Documento Técnico de Desarrollo — Red Antiextorsión Preventiva

1. Configuración técnica del proyecto

La configuración técnica define las tecnologías, herramientas y parámetros base que se utilizarán para desarrollar Red Antiextorsión Preventiva. Se ha planteado una arquitectura web separando frontend, backend y base de datos, permitiendo que cada componente pueda desarrollarse y mantenerse de manera independiente.

1.1. Stack tecnológico definitivo

El sistema utilizará el siguiente stack tecnológico:

Componente	Tecnología	Función
Frontend	Next.js + React + TypeScript	Desarrollo de la interfaz web y experiencia de usuario.
Estilos	Tailwind CSS	Diseño y adaptación responsiva de las interfaces.
Backend	Python + FastAPI	Implementación de la lógica del sistema y exposición de la API REST.
Base de datos	PostgreSQL	Almacenamiento persistente de usuarios, reportes, alertas y demás información del sistema.
ORM	SQLAlchemy	Comunicación entre FastAPI y PostgreSQL mediante modelos de datos.
Migraciones	Alembic	Control de cambios en la estructura de la base de datos.
Autenticación	JWT	Gestión de sesiones y acceso seguro a los recursos de la API.
Contenedores	Docker / Docker Compose	Estandarización del entorno y ejecución de los componentes del sistema.
Control de versiones	Git + GitHub	Gestión del código fuente y seguimiento de cambios.

La comunicación entre el frontend y el backend se realizará mediante una API REST, manteniendo separadas la presentación, la lógica de negocio y la persistencia de datos.

1.2. Versiones utilizadas

Las versiones exactas se registrarán al momento de inicializar los proyectos y generar sus archivos de dependencias. Esto permitirá documentar las versiones realmente utilizadas durante el desarrollo y facilitar la reproducción del entorno.

Como mínimo se registrarán las versiones de:

- Python.
- FastAPI.
- SQLAlchemy.
- PostgreSQL.
- Node.js.
- Next.js.
- React.
- TypeScript.
- Tailwind CSS.
- Docker.

1.3. Herramientas de desarrollo

Para el desarrollo y validación del sistema se utilizarán principalmente:

- Visual Studio Code: entorno principal de programación.
- Git y GitHub: control de versiones y repositorio del proyecto.
- Postman: pruebas manuales de los endpoints de la API.
- Swagger/OpenAPI: documentación y pruebas rápidas de la API proporcionadas por FastAPI.
- Docker Desktop: administración local de los contenedores.
- pgAdmin o herramienta equivalente: inspección y administración de PostgreSQL cuando sea necesario.

Podrán incorporarse herramientas adicionales durante el desarrollo si aportan valor al proceso de implementación.

1.4. Variables de entorno

La configuración sensible o dependiente del entorno no será almacenada directamente en el código fuente. Se utilizarán variables de entorno mediante archivos .env.

Inicialmente se contemplan variables para:

- Conexión con PostgreSQL.
- Clave utilizada para la autenticación JWT.
- Configuración del algoritmo y tiempo de expiración de los tokens.
- URL del backend utilizada por el frontend.
- Configuración del entorno de ejecución.

- Credenciales de servicios externos que puedan incorporarse posteriormente.

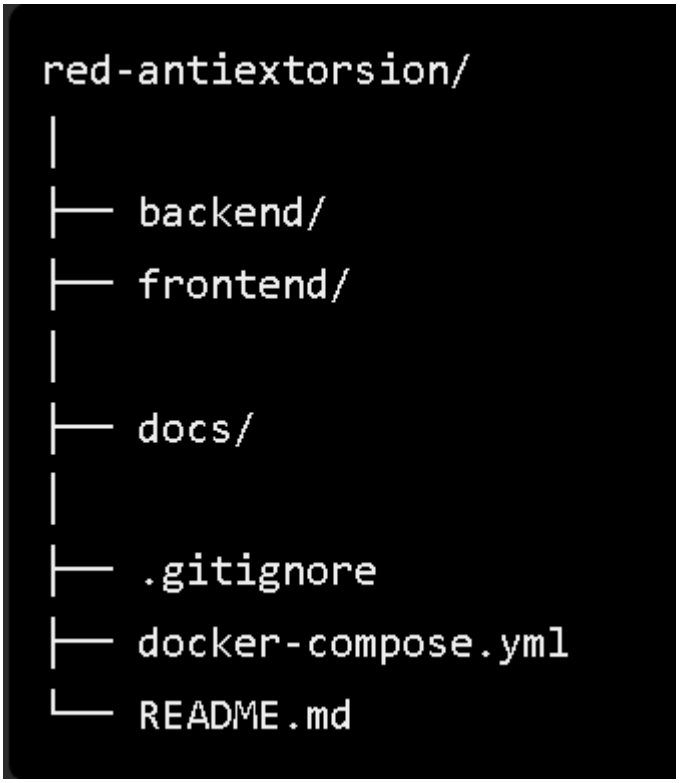
El archivo .env será excluido del repositorio mediante .gitignore. Se podrá mantener un archivo .env.example con los nombres de las variables necesarias, pero sin credenciales ni valores sensibles.

2. Estructura del proyecto

El proyecto se organizará separando el backend, el frontend y los archivos de configuración general. Esta división permitirá mantener responsabilidades claras y facilitará el desarrollo, las pruebas y el despliegue del sistema.

2.1. Estructura general del repositorio

Se utilizará un único repositorio para almacenar los componentes principales del MVP.

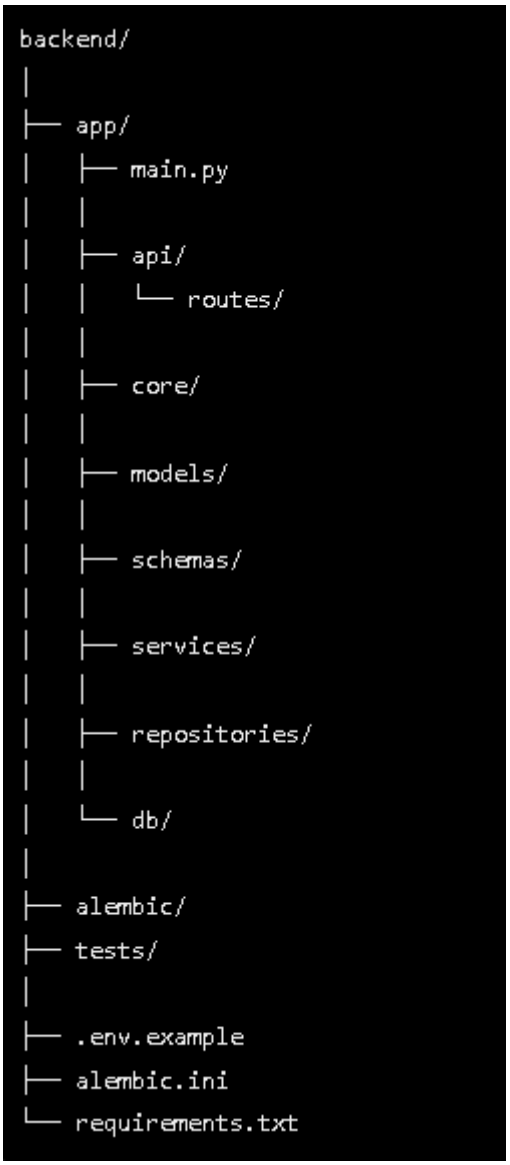


La carpeta backend contendrá la API y la lógica del sistema, mientras que frontend contendrá la aplicación web. docs podrá utilizarse para documentación técnica complementaria relacionada con el desarrollo.

El archivo docker-compose.yml permitirá posteriormente coordinar los servicios necesarios para ejecutar el sistema.

2.2. Estructura del backend

El backend estará desarrollado con FastAPI y organizado por responsabilidades para evitar concentrar toda la lógica en los endpoints.



Las responsabilidades principales serán:

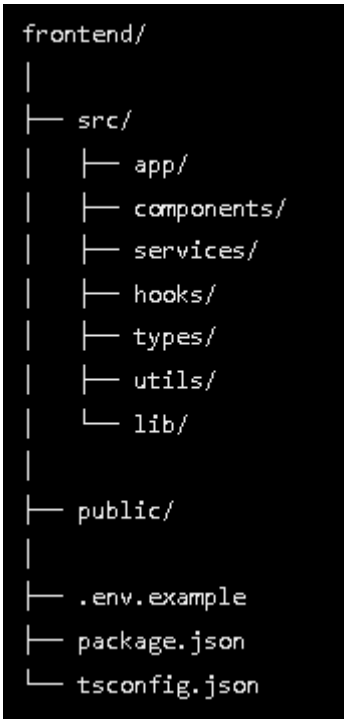
- main.py: punto de entrada de FastAPI.
- api/routes: definición de los endpoints.
- core: configuración general, seguridad y utilidades centrales.
- models: modelos relacionados con las tablas de PostgreSQL.
- schemas: estructuras utilizadas para validar solicitudes y respuestas de la API.
- services: lógica de negocio.
- repositories: acceso y operaciones sobre los datos.
- db: configuración de la conexión y sesión con PostgreSQL.
- alembic: migraciones de la base de datos.
- tests: pruebas del backend.

Esta separación permitirá que los endpoints se mantengan ligeros y deleguen la lógica correspondiente a otras capas.

2.3. Estructura del frontend

El frontend utilizará Next.js, React y TypeScript.

La estructura inicial será:



Las responsabilidades principales serán:

- app: páginas, layouts y rutas de la aplicación.
- components: componentes reutilizables de interfaz.
- services: comunicación con la API del backend.
- hooks: lógica reutilizable mediante hooks de React.
- types: interfaces y tipos de TypeScript.
- utils: funciones auxiliares.
- lib: configuraciones o integraciones utilizadas por diferentes partes de la aplicación.
- public: recursos estáticos.

La estructura podrá ampliarse conforme aparezcan necesidades reales durante el desarrollo.

2.4. Convenciones de nombres y código

Para mantener consistencia durante el desarrollo se utilizarán convenciones propias de cada tecnología.



Reglas generales

Se procurará:

- Utilizar nombres descriptivos.
- Evitar archivos excesivamente grandes.
- Separar la lógica de negocio de la interfaz y de los endpoints.
- No almacenar credenciales dentro del código.
- Mantener commits pequeños y descriptivos.
- Eliminar código que haya quedado sin uso.
- Documentar únicamente aquellas partes cuya lógica no resulte evidente.

Estas convenciones podrán ajustarse durante el desarrollo cuando exista una justificación técnica.

3. Diseño de la base de datos

La base de datos permitirá almacenar y relacionar la información necesaria para el funcionamiento de la Red Antiextorsión Preventiva. El diseño se realizará considerando la naturaleza privada de la plataforma, la protección de información sensible y la necesidad de analizar reportes para identificar patrones y generar alertas preventivas.

3.1. Entidades

A partir de las funcionalidades definidas para el sistema, se identifican inicialmente las siguientes entidades principales:

Organización

- Representa a una empresa, asociación u organización autorizada que forma parte de la red privada. Permitirá agrupar usuarios y controlar el acceso a la información correspondiente.

Usuario

- Representa a una persona autorizada para acceder a la plataforma. Cada usuario estará asociado a una organización y contará con un rol que determinará las acciones que puede realizar dentro del sistema.

Rol

- Define el nivel de acceso y las responsabilidades de los usuarios dentro de la plataforma. Permitirá diferenciar, por ejemplo, usuarios operativos y administradores o moderadores.

Reporte

- Representa un hecho, amenaza, incidente o información relevante registrada dentro de la red. Constituye una de las principales fuentes de información para el análisis preventivo del sistema.

Evidencia

- Representa los archivos o elementos de respaldo asociados a un reporte, como imágenes, documentos u otros recursos permitidos por el sistema. Su registro será opcional.

Ubicación

- Representa la referencia geográfica relacionada con un reporte. Permitirá identificar zonas donde se registran hechos y posteriormente realizar análisis espaciales sin exponer información innecesariamente precisa a usuarios no autorizados.

Indicador

- Representa una señal o característica relevante identificada a partir de la información registrada. Los indicadores servirán como insumo para determinar posibles patrones y niveles de riesgo.

Análisis de riesgo

- Representa el resultado del procesamiento de uno o varios elementos de información disponibles en el sistema. Permitirá registrar el nivel de riesgo obtenido, los indicadores considerados y otra información necesaria para justificar el resultado.

Alerta

- Representa una advertencia preventiva generada a partir del análisis de la información disponible. Las alertas podrán ser consultadas por los usuarios autorizados de acuerdo con su organización y nivel de acceso.
- Estas entidades constituyen el modelo inicial del sistema y podrán ajustarse durante el desarrollo si la implementación evidencia la necesidad de incorporar, dividir o simplificar alguna de ellas.

3.2. Relaciones

Las entidades identificadas se relacionan de acuerdo con las reglas de funcionamiento de la red privada. Las relaciones principales propuestas para el sistema son las siguientes:

Organización — Usuario

Una organización puede tener varios usuarios registrados, mientras que cada usuario pertenece a una organización.

Cardinalidad: Organización 1 : N Usuario

Esta relación permitirá controlar qué usuarios forman parte de cada organización participante de la red.

Rol — Usuario

Un rol puede estar asignado a varios usuarios y cada usuario tendrá un rol principal dentro del sistema.

Cardinalidad: Rol 1 : N Usuario

El rol determinará las operaciones que cada usuario podrá realizar.

Durante la implementación se evaluará si los roles requieren una tabla independiente o pueden representarse mediante un valor enumerado (enum).

Usuario — Reporte

Un usuario autorizado puede registrar varios reportes. Cada reporte tendrá asociado al usuario responsable de su registro.

Cardinalidad: Usuario 1 : N Reporte

La identidad del usuario que registra la información será tratada como información interna y no deberá exponerse innecesariamente a otros usuarios.

Organización — Reporte

Cada reporte estará asociado a la organización desde la cual fue registrado. Una organización podrá registrar múltiples reportes.

Cardinalidad: Organización 1 : N Reporte

Esta relación facilitará el control de acceso y la trazabilidad de la información dentro de la red.

Reporte — Evidencia

Un reporte podrá contener ninguna, una o varias evidencias. Cada evidencia estará asociada a un único reporte.

Cardinalidad: Reporte 1 : 0..N Evidencia

La evidencia será opcional para evitar que su ausencia impida registrar información preventiva relevante.

Ubicación — Reporte

Una ubicación podrá estar relacionada con múltiples reportes, mientras que cada reporte podrá contener una referencia geográfica asociada.

Cardinalidad propuesta: Ubicación 1 : N Reporte

Esta relación será revisada durante el modelo lógico para determinar si Ubicación requiere una entidad independiente o si sus atributos deben formar parte directamente del reporte.

Reporte — Indicador

Un reporte puede presentar varios indicadores y un mismo tipo de indicador puede aparecer en distintos reportes.

Conceptualmente, esta relación es:

Cardinalidad: Reporte N : M Indicador

En el modelo lógico esta relación podrá resolverse mediante una entidad intermedia que permita registrar los indicadores detectados en cada reporte.

Reporte — Análisis de riesgo

Un reporte podrá ser procesado para obtener un análisis de riesgo. El diseño permitirá conservar los resultados de análisis cuando sea necesario.

Cardinalidad inicial: Reporte 1 : N Análisis de riesgo

Esto permitirá mantener trazabilidad si un reporte vuelve a ser evaluado posteriormente debido a nueva información o cambios en las reglas de análisis.

Indicador — Análisis de riesgo

Un análisis de riesgo puede considerar varios indicadores y un indicador puede participar en diferentes análisis.

Cardinalidad: Indicador N : M Análisis de riesgo

Esta relación permitirá conocer qué señales contribuyeron al resultado obtenido y facilitará la explicación del nivel de riesgo generado.

Análisis de riesgo — Alerta

Un análisis que alcance las condiciones definidas por el sistema podrá generar una alerta preventiva.

Cardinalidad: Análisis de riesgo 1 : 0..N Alerta

No todos los análisis necesariamente producirán una alerta.

3.3. Modelo lógico

El modelo lógico transforma las entidades y relaciones identificadas previamente en estructuras que posteriormente serán implementadas en PostgreSQL.

Para el MVP se propone utilizar identificadores UUID en las entidades principales, permitiendo evitar identificadores numéricos consecutivos fácilmente predecibles y facilitando futuras integraciones.

Tabla: organizaciones

Representa las organizaciones autorizadas que participan en la red privada.

Clave primaria: id

Campos principales:

- id
- nombre
- tipo
- estado
- fecha_creacion

Tabla: usuarios

Almacena los usuarios autorizados para acceder al sistema.

Clave primaria: id
Clave foránea: organizacion_id → organizaciones.id

Campos principales:

- id
- organizacion_id
- nombre
- correo
- password_hash
- rol
- estado
- fecha_creacion
- ultimo_acceso

El campo rol se implementará inicialmente mediante un tipo enumerado, evitando una tabla adicional mientras los roles del MVP permanezcan limitados y estables.

Tabla: ubicaciones

Almacena referencias geográficas utilizadas por los reportes.

Clave primaria: id

Campos principales:

- id
- departamento
- provincia
- distrito
- zona_referencia

- latitud
- longitud

La precisión de las coordenadas y su exposición dependerán de las reglas de seguridad definidas para el sistema.

Tabla: reportes

Representa la información preventiva o incidentes registrados por usuarios autorizados.

Clave primaria: id

Claves foráneas:

- usuario_id → usuarios.id
- organizacion_id → organizaciones.id
- ubicacion_id → ubicaciones.id

Campos principales:

- id
- usuario_id
- organizacion_id
- ubicacion_id
- tipo
- descripcion
- fecha_incidente
- fecha_registro
- estado

Tabla: evidencias

Almacena las referencias a archivos asociados a los reportes.

Clave primaria: id
Clave foránea: reporte_id → reportes.id

Campos principales:

- id
- reporte_id
- tipo_archivo
- ruta_archivo

- nombre_archivo
- fecha_creacion

Los archivos no necesariamente serán almacenados directamente dentro de PostgreSQL. La base de datos podrá conservar únicamente la referencia al recurso almacenado.

Tabla: indicadores

Representa los tipos de señales que el sistema puede identificar dentro de la información registrada.

Clave primaria: id

Campos principales:

- id
- nombre
- descripcion
- categoria
- peso
- estado

El campo peso podrá utilizarse posteriormente como parte de las reglas para calcular niveles de riesgo.

Tabla: reporte_indicadores

Resuelve la relación muchos a muchos entre reportes e indicadores.

Clave primaria compuesta: reporte_id + indicador_id

Claves foráneas:

- reporte_id → reportes.id
- indicador_id → indicadores.id

Campos principales:

- reporte_id
- indicador_id
- valor
- confianza
- fecha_deteccion

Esta estructura permitirá registrar no solamente qué indicador fue detectado, sino también información asociada a su detección.

Tabla: analisis_riesgo

Almacena los resultados obtenidos durante el procesamiento de los reportes.

Clave primaria: id
Clave foránea: reporte_id → reportes.id

Campos principales:

- id
- reporte_id
- nivel_riesgo
- puntaje
- justificacion
- version_modelo
- fecha_analisis

Mantener diferentes registros permitirá conservar el historial de análisis realizados sobre un mismo reporte.

Tabla: analisis_indicadores

Relaciona los indicadores considerados específicamente en cada análisis de riesgo.

Clave primaria compuesta: analisis_id + indicador_id

Claves foráneas:

- analisis_id → analisis_riesgo.id
- indicador_id → indicadores.id

Campos principales:

- analisis_id
- indicador_id
- peso_aplicado
- contribucion

Esta relación permitirá mantener trazabilidad sobre las señales que participaron en el cálculo de cada resultado.

Tabla: alertas

Representa las alertas preventivas generadas por el sistema.

Clave primaria: id

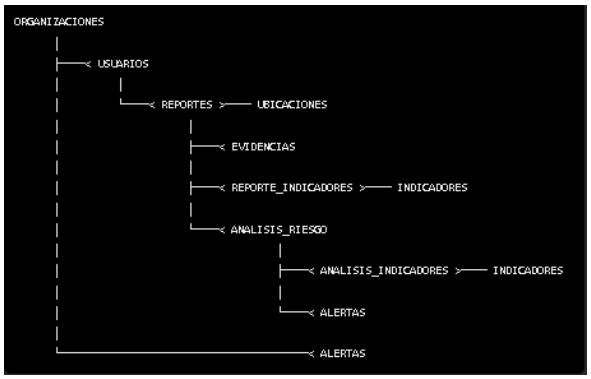
Claves foráneas:

- analisis_id → analisis_riesgo.id
- organizacion_id → organizaciones.id

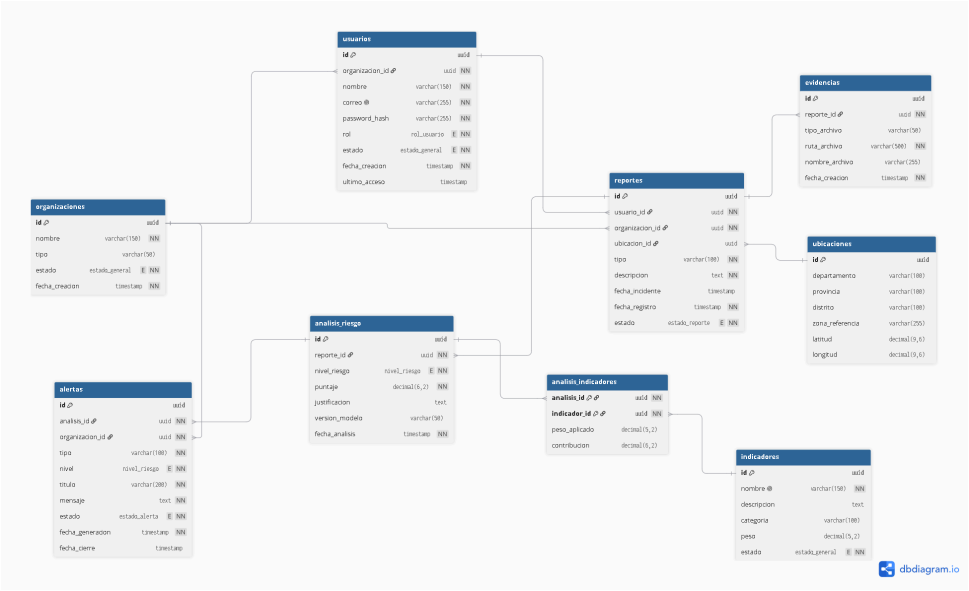
Campos principales:

- id
- analisis_id
- organizacion_id
- tipo
- nivel
- titulo
- mensaje
- estado
- fecha_generacion
- fecha_cierre

Estructura lógica general



Modelo lógico de base de datos — Versión 1.0 (MVP)



El modelo podrá evolucionar durante la implementación mediante migraciones controladas con Alembic cuando se identifiquen nuevos requisitos técnicos o funcionales.

3.4. Diccionario de datos

El diccionario de datos describe la estructura de las tablas definidas para el MVP, especificando los campos principales, tipos de datos, restricciones y propósito de cada atributo.

Los identificadores principales utilizarán UUID. Los campos marcados como NN (NOT NULL) serán obligatorios.

Tabla: organizaciones

Almacena las organizaciones autorizadas que forman parte de la red privada.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único de la organización.
nombre	VARCHAR(150)	NN	Nombre de la organización.
tipo	VARCHAR(50)	Opcional	Tipo de organización participante.
estado	ENUM	NN	Estado de la organización: ACTIVO o INACTIVO.

fecha_creacion	TIMESTAMP	NN	Fecha y hora de registro de la organización.

Tabla: usuarios

Almacena las cuentas autorizadas para acceder a la plataforma.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único del usuario.
organizacion_id	UUID	FK, NN	Organización a la que pertenece el usuario.
nombre	VARCHAR(150)	NN	Nombre del usuario.
correo	VARCHAR(255)	UNIQUE, NN	Correo utilizado para autenticación.
password_hash	VARCHAR(255)	NN	Contraseña almacenada mediante hash seguro.
rol	ENUM	NN	Rol del usuario: USUARIO, MODERADOR o ADMINISTRADOR.
estado	ENUM	NN	Estado de la cuenta: ACTIVO o INACTIVO.
fecha_creacion	TIMESTAMP	NN	Fecha de creación de la cuenta.
ultimo_acceso	TIMESTAMP	Opcional	Fecha y hora del último acceso registrado.

Tabla: ubicaciones

Almacena las referencias geográficas asociadas a los reportes.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único de la ubicación.
departamento	VARCHAR(100)	Opcional	Departamento correspondiente al incidente.
provincia	VARCHAR(100)	Opcional	Provincia correspondiente.
distrito	VARCHAR(100)	Opcional	Distrito donde se produjo o reportó el hecho.

zona_referencia	VARCHAR(255)	Opcional	Referencia general de la zona.
latitud	DECIMAL(9,6)	Opcional	Coordenada geográfica de latitud.
longitud	DECIMAL(9,6)	Opcional	Coordenada geográfica de longitud.

La precisión y exposición de la ubicación deberá controlarse según los permisos del usuario y las políticas de privacidad del sistema.

Tabla: reportes

Almacena los hechos o situaciones registrados por los usuarios autorizados.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único del reporte.
usuario_id	UUID	FK, NN	Usuario responsable del registro.
organizacion_id	UUID	FK, NN	Organización propietaria del reporte.
ubicacion_id	UUID	FK, opcional	Ubicación relacionada con el hecho.
tipo	VARCHAR(100)	NN	Clasificación general del reporte.
descripcion	TEXT	NN	Descripción del hecho registrado.
fecha_incidente	TIMESTAMP	Opcional	Fecha y hora aproximada en que ocurrió el hecho.
fecha_registro	TIMESTAMP	NN	Fecha y hora en que se registró el reporte.
estado	ENUM	NN	REGISTRADO, EN_ANALISIS, ANALIZADO o CERRADO.

Tabla: evidencias

Almacena referencias a los recursos utilizados como respaldo de un reporte.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único de la evidencia.
reporte_id	UUID	FK, NN	Reporte al que pertenece la evidencia.
tipo_archivo	VARCHAR(50)	Opcional	Tipo o formato del recurso asociado.
ruta_archivo	VARCHAR(500)	NN	Ubicación o referencia del archivo almacenado.
nombre_archivo	VARCHAR(255)	Opcional	Nombre original o identificador visible del archivo.
fecha_creacion	TIMESTAMP	NN	Fecha en que se registró la evidencia.

La base de datos almacenará principalmente la referencia al archivo y no necesariamente su contenido binario.

Tabla: indicadores

Contiene el catálogo de señales utilizadas durante el análisis preventivo.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único del indicador.
nombre	VARCHAR(150)	UNIQUE, NN	Nombre del indicador.
descripcion	TEXT	Opcional	Explicación del indicador.
categoria	VARCHAR(100)	Opcional	Clasificación del indicador.
peso	DECIMAL(5,2)	Opcional	Peso base utilizado para el cálculo de riesgo.
estado	ENUM	NN	ACTIVO o INACTIVO.

Tabla: analisis_riesgo

Almacena los resultados de los análisis realizados sobre los reportes.

Campo	Tipo	Restricción	Descripción

id	UUID	PK	Identificador único del análisis.
reporte_id	UUID	FK, NN	Reporte analizado.
nivel_riesgo	ENUM	NN	Nivel obtenido: BAJO, MODERADO, ALTO o CRITICO.
puntaje	DECIMAL(6,2)	NN	Valor numérico resultante del análisis.
justificacion	TEXT	Opcional	Explicación del resultado obtenido.
version_modelo	VARCHAR(50)	Opcional	Versión de las reglas o modelo utilizado para realizar el análisis.
fecha_analisis	TIMESTAMP	NN	Fecha y hora de ejecución del análisis.

La existencia de múltiples registros para un mismo reporte permitirá conservar el historial de evaluaciones realizadas.

Tabla: analisis_indicadores

Registra los indicadores considerados en cada análisis de riesgo y permite conocer su participación en el resultado.

Campo	Tipo	Restricción	Descripción
analisis_id	UUID	PK, FK, NN	Análisis de riesgo asociado.
indicador_id	UUID	PK, FK, NN	Indicador detectado o utilizado.
peso_aplicado	DECIMAL(5,2)	Opcional	Peso utilizado para el indicador durante ese análisis.
contribucion	DECIMAL(6,2)	Opcional	Contribución del indicador al resultado obtenido.

La combinación analisis_id + indicador_id constituye la clave primaria compuesta.

Tabla: alertas

Almacena las advertencias preventivas generadas como consecuencia de los análisis realizados.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único de la alerta.
analisis_id	UUID	FK, NN	Análisis que originó la alerta.
organizacion_id	UUID	FK, NN	Organización a la que se encuentra asociada la alerta.
tipo	VARCHAR(100)	NN	Clasificación de la alerta.
nivel	ENUM	NN	Nivel de riesgo asociado a la alerta.
título	VARCHAR(200)	NN	Título resumido de la alerta.
mensaje	TEXT	NN	Contenido preventivo de la alerta.
estado	ENUM	NN	ACTIVA, ATENDIDA o CERRADA.
fecha_generacion	TIMESTAMP	NN	Fecha y hora de generación.
fecha_cierre	TIMESTAMP	Opcional	Fecha y hora en que la alerta fue cerrada.

Tipos enumerados

Para mantener consistencia se utilizarán inicialmente los siguientes valores controlados:

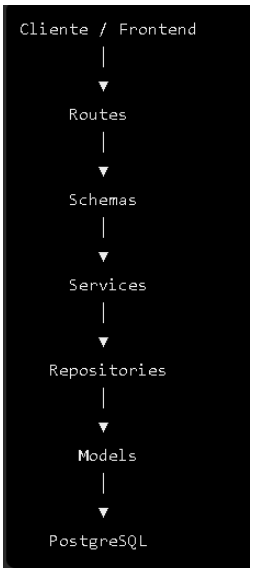
Enumeración	Valores
estado_general	ACTIVO, INACTIVO
rol_usuario	USUARIO, MODERADOR, ADMINISTRADOR
estado_reporte	REGISTRADO, EN_ANALISIS, ANALIZADO, CERRADO
nivel_riesgo	BAJO, MODERADO, ALTO, CRITICO
estado_alerta	ACTIVA, ATENDIDA, CERRADA

El presente diccionario corresponde al modelo lógico v1.0 del MVP. Durante la implementación podrán incorporarse restricciones, índices u otros ajustes mediante migraciones controladas, manteniendo actualizada esta documentación.

4. Diseño del backend

El backend de Red Antiextorsión Preventiva será desarrollado con FastAPI y seguirá una arquitectura en capas, con el objetivo de separar las responsabilidades relacionadas con la exposición de la API, la lógica de negocio y el acceso a datos.

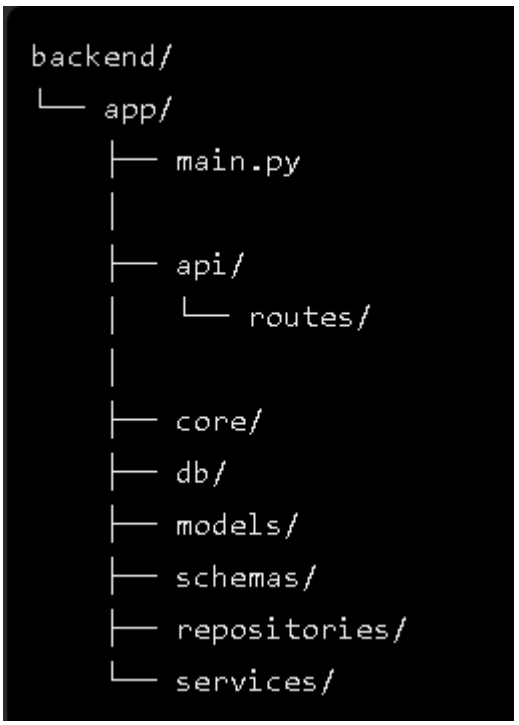
Se utilizarán adicionalmente los patrones Service Layer y Repository, evitando concentrar la lógica del sistema directamente en los endpoints.



Cada capa tendrá una responsabilidad específica y deberá evitar asumir funciones correspondientes a otras capas.

4.1. Organización de FastAPI

La aplicación FastAPI estará organizada inicialmente de la siguiente manera:



main.py

Será el punto de entrada de la aplicación.

Tendrá como responsabilidades principales:

- Inicializar FastAPI.
- Registrar los routers.
- Configurar middleware cuando sea necesario.
- Definir configuraciones generales relacionadas con la aplicación.

No deberá contener lógica de negocio.

api/routes

Contendrá los endpoints expuestos mediante la API REST.

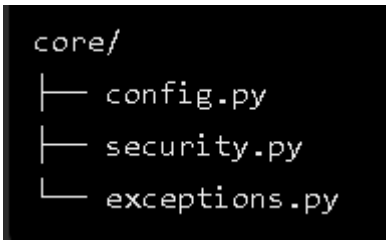
Los archivos estarán agrupados según los recursos principales del sistema, por ejemplo:



Los endpoints serán responsables principalmente de recibir solicitudes, validar los datos mediante los esquemas correspondientes, invocar los servicios y devolver respuestas HTTP.

core

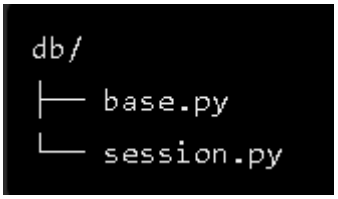
Contendrá funcionalidades transversales utilizadas por diferentes módulos.



Aquí podrán ubicarse la configuración mediante variables de entorno, funciones relacionadas con JWT, hashing de contraseñas y excepciones generales.

db

Contendrá la configuración necesaria para trabajar con PostgreSQL y SQLAlchemy.

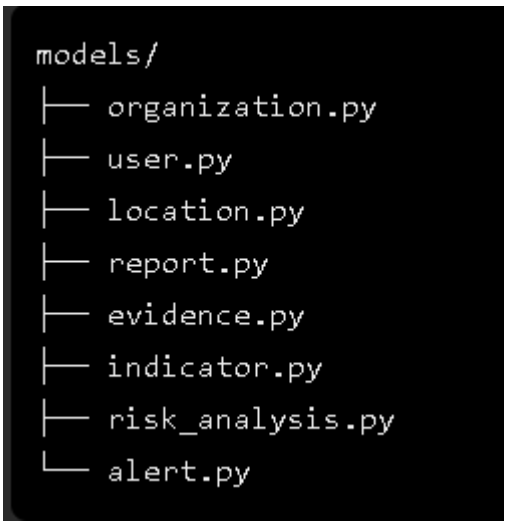


Su responsabilidad será proporcionar la configuración y las sesiones necesarias para acceder a la base de datos.

4.2. Modelos

Los modelos representarán las estructuras persistentes almacenadas en PostgreSQL.

Se implementarán utilizando SQLAlchemy y corresponderán directamente con las tablas definidas en el modelo lógico de la base de datos.



Las relaciones necesarias entre modelos serán definidas mediante las capacidades proporcionadas por SQLAlchemy.

Los modelos tendrán como responsabilidad representar la persistencia y no contendrán la lógica principal del negocio.

4.3. Esquemas

Los esquemas serán implementados mediante Pydantic y permitirán validar la información recibida y controlar los datos devueltos por la API.

Se podrán definir diferentes esquemas según la operación.



AlertResponse

Esta separación permitirá evitar que los modelos internos de la base de datos sean expuestos directamente mediante la API.

Los esquemas permitirán:

- Validar tipos de datos.

- Definir campos obligatorios y opcionales.
- Controlar la estructura de las respuestas.
- Evitar la exposición accidental de información sensible.

Por ejemplo, password_hash nunca deberá formar parte de una respuesta destinada al frontend.

4.4. Servicios

La capa de servicios contendrá la lógica de negocio del sistema.

Los servicios recibirán información previamente validada y coordinarán las operaciones necesarias para completar cada caso de uso.

Inicialmente podrán existir servicios como:



Entre sus responsabilidades se encontrarán:

- Aplicar reglas de negocio.
- Coordinar operaciones entre diferentes repositorios.
- Ejecutar el procesamiento de reportes.
- Coordinar la evaluación de indicadores.
- Calcular niveles de riesgo.
- Determinar cuándo debe generarse una alerta.

La lógica relacionada con el análisis preventivo deberá permanecer principalmente en esta capa y no dentro de los endpoints.

4.5. Repositorios

La capa Repository encapsula las operaciones relacionadas con la persistencia de información.

```
repositories/
├── user_repository.py
├── organization_repository.py
├── report_repository.py
├── indicator_repository.py
├── risk_repository.py
└── alert_repository.py
```

Los repositorios realizarán operaciones como:

- Crear registros.
- Consultar información.
- Actualizar registros.
- Aplicar filtros.
- Ejecutar consultas específicas sobre PostgreSQL.

El repositorio no deberá decidir si un reporte representa riesgo alto o si debe generarse una alerta. Esas decisiones corresponden a la capa de servicios.

4.6. Manejo de errores

El backend utilizará un manejo centralizado y consistente de errores.

Se diferenciarán principalmente:

- Errores de validación.
- Recursos no encontrados.
- Acceso no autorizado.
- Operaciones prohibidas.
- Conflictos de información.
- Errores internos inesperados.

Las respuestas deberán utilizar códigos HTTP apropiados, por ejemplo:

Código	Uso general
400	Solicitud incorrecta o regla de negocio inválida.
401	Usuario no autenticado o credenciales inválidas.

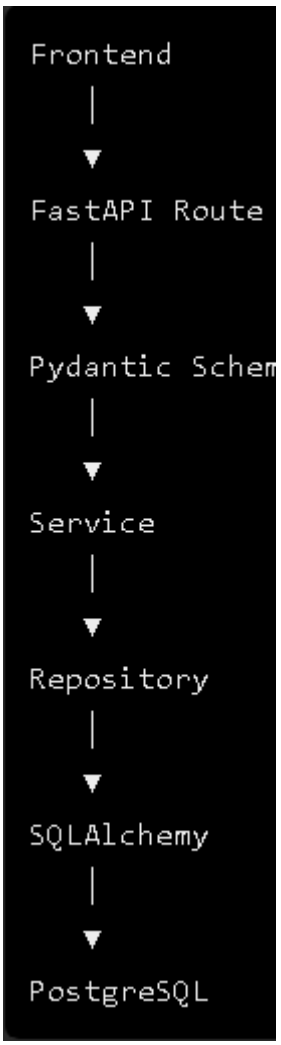
403	Usuario autenticado sin permisos suficientes.
404	Recurso solicitado no encontrado.
409	Conflicto con información existente.
422	Datos que no cumplen con la validación requerida.
500	Error interno no controlado.

Los detalles técnicos internos, consultas SQL, credenciales, tokens, rutas del servidor o trazas de ejecución no deberán ser enviados al cliente.

Los errores internos deberán registrarse mediante mecanismos de logging para facilitar su diagnóstico sin exponer información sensible.

Flujo general del backend

De forma simplificada, una solicitud seguirá el siguiente recorrido:

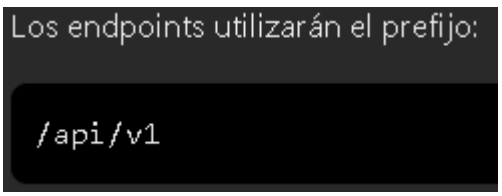


Cuando una operación requiera lógica adicional, el servicio podrá coordinar diferentes repositorios y otros servicios sin trasladar dicha responsabilidad a la capa de rutas.

Esta organización permitirá mantener un backend modular y facilitará las pruebas, el mantenimiento y la incorporación progresiva de nuevas funcionalidades durante el desarrollo del MVP.

5. Diseño de la API REST

El backend expondrá sus funcionalidades mediante una API REST desarrollada con FastAPI. Esta API constituirá el principal mecanismo de comunicación entre el frontend y la lógica del sistema.



La versión incluida en la URL permitirá evolucionar la API posteriormente sin afectar directamente a los clientes existentes.

Las solicitudes y respuestas utilizarán principalmente JSON, excepto aquellas operaciones relacionadas con carga de archivos.

Los recursos protegidos requerirán autenticación mediante un token JWT enviado en la cabecera:

```
Authorization: Bearer <token>
```

5.1. Autenticación

La autenticación permitirá validar la identidad de los usuarios autorizados de la red privada.



Devuelve información básica del usuario correspondiente al token utilizado.

Para el MVP, la creación pública de cuentas no estará habilitada. Los usuarios serán creados únicamente mediante operaciones autorizadas dentro de la plataforma.

5.2. Usuarios y roles

Las operaciones relacionadas con usuarios estarán restringidas según los permisos establecidos.

Listar usuarios

```
GET /api/v1/users
```

Permite consultar los usuarios disponibles según la organización y los permisos del solicitante.

Obtener usuario

```
GET /api/v1/users/{user_id}
```

Crear usuario

```
POST /api/v1/users
```

Operación disponible únicamente para perfiles autorizados.

Actualizar usuario

```
PATCH /api/v1/users/{user_id}
```

Permitirá modificar únicamente los campos autorizados.

Actualizar usuario

```
PATCH /api/v1/users/{user_id}
```

Permitirá modificar únicamente los campos autorizados.

Cambiar estado del usuario

```
PATCH /api/v1/users/{user_id}/status
```

Permitirá activar o desactivar una cuenta sin eliminar su historial.

Los roles contemplados inicialmente serán:

```
USUARIO
MODERADOR
ADMINISTRADOR
```

5.3. Reportes

Los reportes constituyen una de las principales fuentes de información del sistema.

Registrar reporte

```
POST /api/v1/reports
```

Permitirá registrar un nuevo hecho o situación relevante.

El backend asociará automáticamente el usuario autenticado y su organización al reporte, evitando que estos identificadores sean definidos libremente desde el frontend.

Listar reportes

```
GET /api/v1/reports
```

Permitirá consultar reportes de acuerdo con los permisos del usuario.

Podrán incorporarse filtros mediante parámetros de consulta, por ejemplo:

```
?estado=ANALIZADO
?tipo=AMENAZA
?nivel_riesgo=ALTO
```

Obtener reporte

```
GET /api/v1/reports/{report_id}
```

Devuelve el detalle de un reporte autorizado.

Actualizar reporte

```
PATCH /api/v1/reports/{report_id}
```

Permitirá modificar únicamente información autorizada y mientras el estado del reporte lo permita.

Adjuntar evidencia

```
POST /api/v1/reports/{report_id}/evidence
```

Permitirá asociar una evidencia a un reporte.

La carga utilizará `multipart/form-data` cuando se envíen archivos.

5.4. Alertas

Las alertas permitirán comunicar situaciones que requieran atención preventiva.

Listar alertas

```
GET /api/v1/alerts
```

Podrá admitir filtros como:

```
?estado=ACTIVA
?nivel=ALTO
```

Obtener alerta

```
GET /api/v1/alerts/{alert_id}
```

Actualizar estado de alerta

```
PATCH /api/v1/alerts/{alert_id}/status
```

Permitirá realizar transiciones autorizadas, por ejemplo:

```
ACTIVA → ATENDIDA → CERRADA
```

La generación de alertas será responsabilidad de la lógica interna del sistema y no de un endpoint público utilizado libremente por los usuarios.

5.5. Análisis de riesgo

Este módulo expondrá los resultados del procesamiento preventivo.

Consultar análisis de un reporte

```
GET /api/v1/reports/{report_id}/risk-analysis
```

Permitirá obtener los análisis realizados sobre un reporte, incluyendo:

- Nivel de riesgo.
- Puntaje.
- Justificación.
- Indicadores detectados.
- Fecha del análisis.
- Versión del mecanismo utilizado.

Ejecutar o repetir análisis

```
POST /api/v1/reports/{report_id}/risk-analysis
```

Esta operación estará restringida a procesos internos o perfiles autorizados.

Permitirá ejecutar nuevamente el análisis cuando sea necesario conservar una nueva evaluación del reporte.

5.6. Estadísticas

La API proporcionará información agregada necesaria para el panel preventivo.

Resumen general

```
GET /api/v1/statistics/summary
```

Podrá proporcionar datos como:

- Cantidad de reportes.
- Reportes por nivel de riesgo.
- Alertas activas.
- Distribución de reportes por estado.

Estadísticas geográficas

```
GET /api/v1/statistics/geographic
```

Permitirá obtener información agregada por distrito o zona para apoyar la representación del riesgo en el mapa.

Indicadores frecuentes

```
GET /api/v1/statistics/indicators
```

Permitirá consultar los indicadores detectados con mayor frecuencia durante los análisis.

Las estadísticas deberán respetar las reglas de autorización y privacidad definidas para la red.

5.7. Documentación de endpoints

FastAPI permitirá generar automáticamente documentación interactiva basada en OpenAPI.

Durante el desarrollo estarán disponibles principalmente:

/docs

/redoc

Swagger UI será utilizado para realizar pruebas rápidas y revisar:

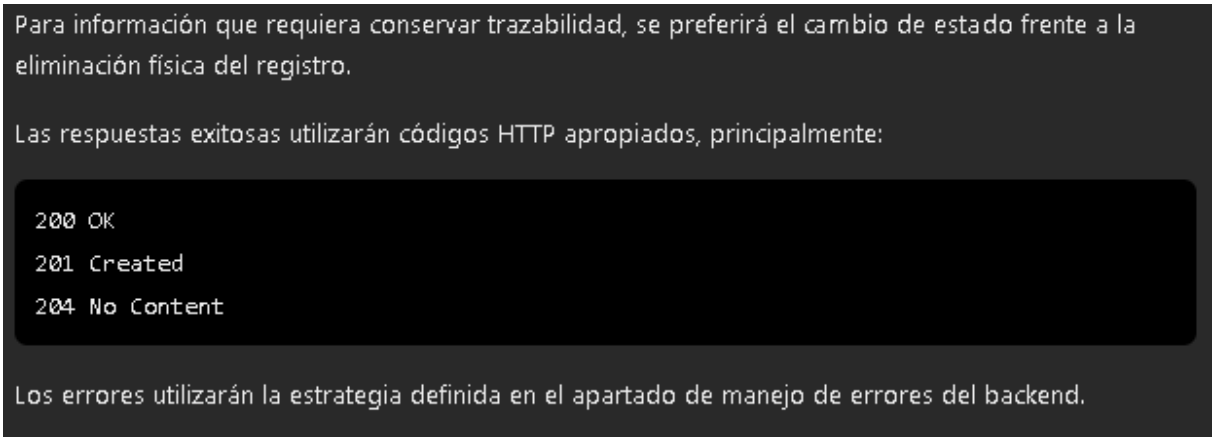
- Métodos HTTP.
- Parámetros.
- Esquemas de entrada.
- Esquemas de respuesta.
- Códigos HTTP.
- Requisitos de autenticación.

Cada endpoint deberá definir correctamente sus esquemas de entrada y salida y los códigos HTTP correspondientes.

Convenciones generales

La API utilizará principalmente:

Método	Uso
GET	Consultar información.
POST	Crear recursos o ejecutar operaciones.
PATCH	Modificar parcialmente un recurso.
DELETE	Eliminar recursos únicamente cuando sea necesario y seguro.



6. Seguridad

La seguridad del sistema se implementará considerando que la plataforma maneja información sensible relacionada con reportes de posibles amenazas y extorsiones dentro de una red privada.

6.1. Autenticación JWT

El acceso a la API estará protegido mediante JSON Web Tokens (JWT). El usuario iniciará sesión con sus credenciales y recibirá un token que deberá utilizar para acceder a los recursos protegidos.

Las contraseñas nunca serán almacenadas directamente, sino mediante algoritmos seguros de hashing.

6.2. Autorización por roles

El sistema utilizará inicialmente tres roles:

- USUARIO
- MODERADOR
- ADMINISTRADOR

Los endpoints verificarán tanto la identidad del usuario como los permisos correspondientes a su rol.

6.3. Protección de información sensible

Los usuarios únicamente podrán acceder a la información permitida dentro de la red y según su organización.

No se expondrán mediante la API contraseñas, hashes, tokens, información interna del servidor ni datos sensibles innecesarios.

Las evidencias y ubicaciones tendrán controles adicionales de acceso.

6.4. Validación de datos

Las solicitudes recibidas por la API serán validadas mediante esquemas Pydantic.

También se aplicarán reglas de negocio para impedir operaciones inválidas, manipulación de identificadores y acceso a recursos pertenecientes a organizaciones no autorizadas.

6.5. Controles contra abuso

El sistema incorporará progresivamente controles para reducir usos indebidos, tales como:

- Restricción de operaciones según rol.
- Validación de archivos y evidencias.
- Control del acceso entre organizaciones.
- Registro de errores y operaciones relevantes.
- Limitación de solicitudes cuando resulte necesario.

La seguridad se implementará de manera incremental durante el desarrollo del MVP, priorizando autenticación, autorización y aislamiento de la información.

7. Diseño técnico del frontend

El frontend será desarrollado con Next.js, React y TypeScript, utilizando Tailwind CSS para la construcción de las interfaces. Su función será proporcionar una interfaz web responsiva para que los usuarios autorizados interactúen con los servicios expuestos por la API REST.

7.1. Organización de Next.js

Se utilizará el App Router de Next.js y una organización modular dentro de src.



Las rutas y layouts estarán ubicados en app, mientras que la lógica reutilizable se mantendrá separada de los componentes visuales.

7.2. Componentes

La interfaz será dividida en componentes reutilizables para evitar duplicación y facilitar su mantenimiento.

Se contemplarán componentes relacionados con:

- Navegación.
- Formularios.
- Reportes.
- Alertas.
- Indicadores de riesgo.
- Estadísticas y visualizaciones.
- Elementos comunes de interfaz.

Las páginas coordinarán estos componentes sin concentrar innecesariamente lógica de negocio.

7.3. Servicios de consumo de API

La comunicación con FastAPI se centralizará en la capa services.



Estos servicios encapsularán las solicitudes HTTP hacia /api/v1, evitando realizar llamadas dispersas directamente desde los componentes.

7.4. Manejo de sesión

Después de una autenticación válida, el frontend mantendrá la sesión necesaria para consumir los recursos protegidos.

Las solicitudes autenticadas enviarán el token correspondiente al backend, mientras que FastAPI será responsable de validar la identidad, organización y permisos del usuario.

La implementación evitará utilizar información almacenada únicamente en el cliente como mecanismo de autorización.

7.5. Manejo de estado

Se utilizará principalmente el estado proporcionado por React para información local de los componentes.

El estado compartido se incorporará únicamente cuando sea necesario para elementos como:

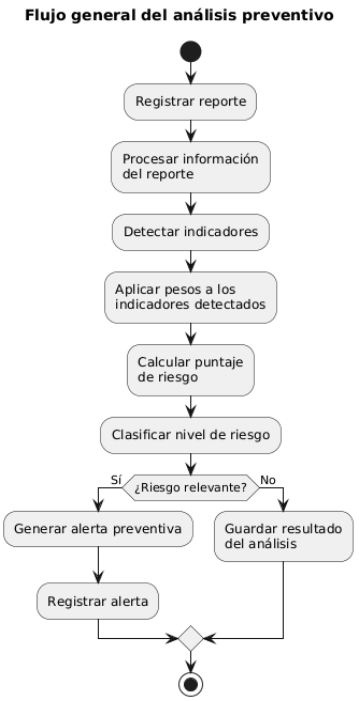
- Usuario autenticado.
- Sesión.
- Información utilizada entre diferentes vistas.

Se evitará incorporar herramientas adicionales de gestión global de estado mientras la complejidad del MVP no las requiera.

8. Lógica de análisis preventivo

La lógica de análisis preventivo permitirá transformar la información registrada en los reportes en indicadores, niveles de riesgo y alertas, con el objetivo de apoyar la identificación temprana de posibles situaciones de amenaza.

Para el MVP se implementará inicialmente un mecanismo basado en reglas e indicadores ponderados, manteniendo una estructura que permita incorporar posteriormente técnicas de análisis más avanzadas.



8.1. Procesamiento de reportes

Cuando un reporte sea registrado, el sistema procesará la información disponible para identificar elementos relevantes para el análisis.

Se considerarán datos como:

- Tipo de reporte.
- Descripción.
- Ubicación.
- Fecha del incidente.
- Información histórica disponible.

El procesamiento generará la información necesaria para evaluar posteriormente los indicadores de riesgo.

8.2. Indicadores

Los indicadores representarán señales que pueden aumentar o modificar el nivel de riesgo de un reporte.

Cada indicador podrá tener un peso, utilizado posteriormente durante el cálculo del riesgo.

Los indicadores utilizados por el MVP serán definidos y ajustados durante la implementación según los escenarios preventivos que se decida demostrar.

8.3. Correlación de información

Cuando exista información suficiente, el sistema podrá relacionar reportes mediante características comunes, tales como:

- Zona.
- Periodo de tiempo.
- Tipo de hecho.
- Indicadores coincidentes.

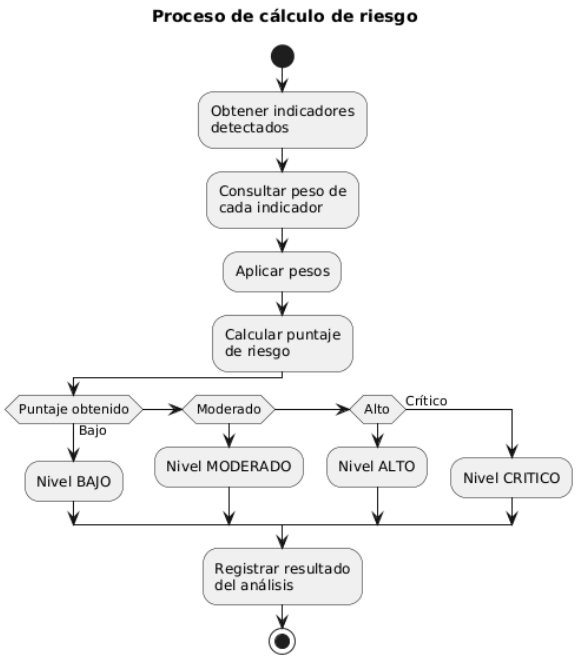
Esta correlación permitirá detectar posibles concentraciones o patrones relevantes.

Para el MVP se priorizarán correlaciones simples y explicables, evitando incorporar algoritmos complejos que no sean necesarios para demostrar el funcionamiento preventivo del sistema.

8.4. Cálculo de riesgo

El sistema calculará un puntaje considerando los indicadores identificados y sus respectivos pesos.

De manera conceptual:



Nivel de riesgo

El resultado será clasificado inicialmente en:

- BAJO
- MODERADO
- ALTO
- CRITICO

Los valores exactos, pesos y umbrales serán definidos durante la implementación y podrán ajustarse sin modificar la arquitectura general del sistema.

Cada análisis almacenará su resultado, justificación y versión del mecanismo utilizado, permitiendo mantener trazabilidad sobre cómo se obtuvo la evaluación.

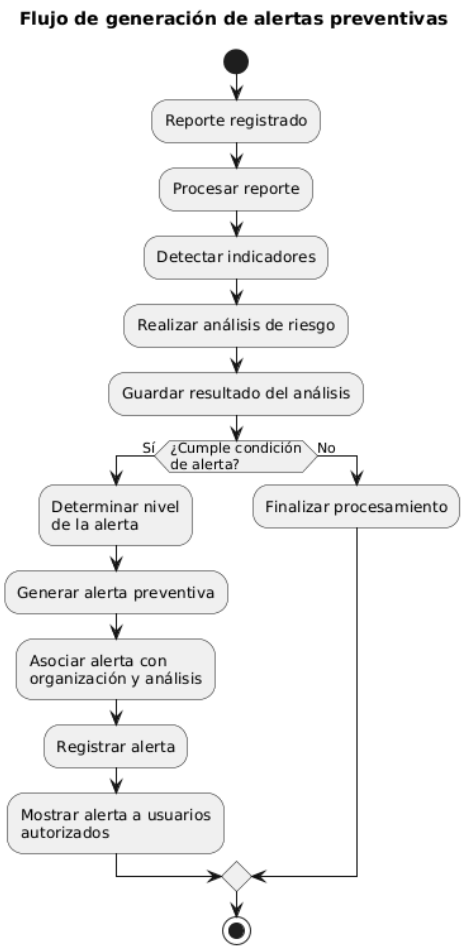
8.5. Generación de alertas

Cuando el análisis determine que una situación cumple las condiciones establecidas para generar una advertencia preventiva, el sistema creará una alerta.

La alerta estará asociada al análisis y a la organización correspondiente.

Podrá contener:

- Tipo de alerta.
- Nivel de riesgo.
- Título.
- Mensaje preventivo.
- Fecha de generación.
- Estado de atención.



La finalidad del análisis será apoyar la prevención y priorización de información, no determinar automáticamente la existencia de un delito ni sustituir la evaluación realizada por las autoridades o responsables correspondientes.

9. Pruebas

Durante el desarrollo del MVP se realizarán pruebas orientadas a verificar el correcto funcionamiento de los componentes principales del sistema.

Se priorizarán los flujos críticos: autenticación, registro de reportes, análisis de riesgo, generación de alertas y control de acceso.

9.1. Pruebas unitarias

Las pruebas unitarias estarán orientadas principalmente a validar la lógica de negocio implementada en la capa de servicios.

Se priorizarán funciones relacionadas con:

- Autenticación y seguridad.

- Procesamiento de reportes.
- Detección de indicadores.
- Cálculo del nivel de riesgo.
- Generación de alertas.
- Validación de reglas de negocio.

Estas pruebas permitirán comprobar el comportamiento de componentes específicos de manera aislada.

9.2. Pruebas de API

Los endpoints principales serán verificados mediante las herramientas proporcionadas por FastAPI/OpenAPI (Swagger UI) y, cuando sea necesario, mediante Postman.

Se comprobarán principalmente:

- Solicitudes válidas e inválidas.
- Códigos de respuesta HTTP.
- Validación de datos.
- Autenticación mediante JWT.
- Restricciones según roles.
- Acceso únicamente a información autorizada.
- Respuestas generadas por los endpoints.

Se dará prioridad a los endpoints que formen parte del flujo principal del MVP.

9.3. Pruebas de integración

Se validarán especialmente los siguientes flujos:

1. Inicio de sesión y acceso a recursos protegidos.
2. Registro y consulta de reportes.
3. Procesamiento de un reporte y generación de su análisis.
4. Detección de indicadores y cálculo del nivel de riesgo.
5. Generación y consulta de alertas.
6. Visualización de información desde el frontend.

Los resultados reales de las pruebas, errores encontrados y correcciones realizadas se registrarán progresivamente durante la implementación.

El objetivo para el MVP será garantizar principalmente la estabilidad de los flujos críticos utilizados durante la demostración, priorizando su correcto funcionamiento sobre una cobertura exhaustiva de pruebas.

10. Despliegue

El MVP será preparado para ejecutarse de manera reproducible utilizando Docker y variables de entorno para separar la configuración del código fuente.

Durante la etapa final del desarrollo se definirá el entorno de alojamiento utilizado para la demostración.

10.1. Docker

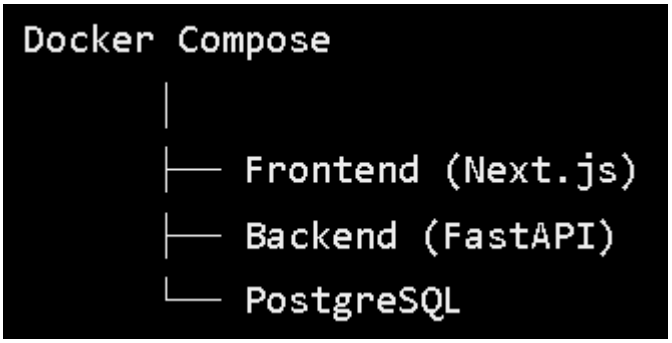
Se utilizará Docker para contenerizar los principales servicios del sistema y facilitar su ejecución en diferentes entornos.

Inicialmente se contemplarán:

- Frontend
- Backend
- PostgreSQL

Durante el desarrollo local, Docker Compose permitirá coordinar los servicios necesarios desde una configuración común.

De manera conceptual:



Las credenciales y configuraciones sensibles serán proporcionadas mediante variables de entorno y no estarán incluidas directamente en las imágenes ni en el repositorio.

10.2. PostgreSQL

PostgreSQL será utilizado como sistema de persistencia principal.

La conexión será configurada mediante variables de entorno y la evolución de la estructura de la base de datos será administrada mediante Alembic.

Esto permitirá reproducir las modificaciones realizadas al esquema durante el desarrollo.

10.3. Backend

FastAPI será preparado para ejecutarse como un servicio independiente.

El backend será responsable de:

- Exponer la API REST.
- Gestionar autenticación y autorización.
- Ejecutar la lógica de negocio.
- Procesar los análisis preventivos.
- Acceder a PostgreSQL.

Su configuración dependerá del entorno mediante variables como la conexión a la base de datos, configuración JWT y orígenes permitidos.

10.4. Frontend

La aplicación Next.js será desplegada independientemente del backend.

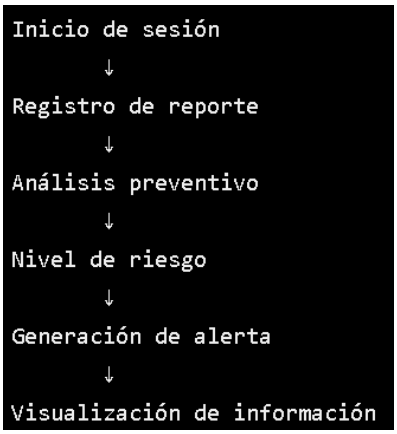
El frontend utilizará una variable de entorno para identificar la dirección de la API:

NEXT_PUBLIC_API_URL

Esto permitirá utilizar diferentes direcciones para desarrollo local y para el entorno de demostración sin modificar el código fuente.

10.5. Entorno de demostración

Para la presentación del MVP se dispondrá de un entorno que permita demostrar el flujo principal:



Antes de la presentación se verificará que el entorno utilizado permita ejecutar correctamente este flujo.

La plataforma específica de alojamiento y las direcciones definitivas serán registradas una vez realizado el despliegue.

Como respaldo para la demostración, el proyecto mantendrá la posibilidad de ejecutarse localmente mediante Docker Compose en caso de problemas externos de conectividad o disponibilidad del servicio desplegado.

11. Registro de implementación

Esta sección permitirá registrar la evolución real del MVP durante su desarrollo.

Se actualizará progresivamente conforme se implementen funcionalidades, se realicen ajustes técnicos y se resuelvan problemas encontrados durante la construcción del sistema.

11.1. Funcionalidades completadas

Se registrarán únicamente las funcionalidades que hayan sido implementadas y verificadas.

Fecha	Funcionalidad	Estado
—	—	—

Ejemplos de funcionalidades que posteriormente podrán registrarse:

- Autenticación.
- Gestión de usuarios.
- Registro de reportes.
- Análisis de riesgo.
- Generación de alertas.
- Dashboard preventivo.
- Integración frontend-backend.

11.2. Cambios realizados

Se documentarán los cambios relevantes realizados respecto al diseño inicial.

Fecha	Cambio	Motivo
—	—	—

No será necesario registrar modificaciones menores de código, sino aquellas que afecten decisiones técnicas, estructura, base de datos o funcionamiento del sistema.

11.3. Problemas encontrados

Se registrarán los problemas técnicos relevantes encontrados durante la implementación y la solución aplicada.

Fecha	Problema	Solución
—	—	—

Esto permitirá mantener trazabilidad sobre dificultades importantes surgidas durante el desarrollo.

11.4. Decisiones técnicas

Se registrarán decisiones tomadas durante la implementación que modifiquen o complementen lo definido inicialmente.

Fecha	Decisión	Justificación
—	—	—

Entre ellas podrán encontrarse cambios de librerías, modificaciones del modelo de datos, ajustes del algoritmo de riesgo o decisiones relacionadas con el despliegue.

11.5. Pendientes

Se mantendrá una lista breve de tareas técnicas todavía necesarias para completar el MVP.

Pendiente	Prioridad	Estado
—	—	—

Los pendientes se clasificarán principalmente como:

- Alta: necesarios para el funcionamiento o demostración del MVP.
- Media: importantes, pero no bloquean el flujo principal.
- Baja: mejoras que podrán realizarse si el tiempo disponible lo permite.

Este registro se mantendrá actualizado durante la implementación y servirá como evidencia de la evolución técnica del proyecto.